

Automation Engineer Technical Homework

Show your ability to design and implement a lightweight automation workflow that uses **agentic orchestration** principles. The goal is to move beyond simple data processing and build an interactive agent that can reason over a dataset.

Parts 1 and 2 of the assignment are required.

Part 3 of the assessment is a bonus (for overachievers ;-))

Part 1: Agentic Workflow Design (Written + Diagram)

Your task is to design a system that ingests sales leads, indexes them for semantic search, and makes them available for querying through a conversational agent. 🤖

Tasks:

1. Provide a simple **workflow diagram** (e.g., using Mermaid, draw.io, or a sketch) that illustrates the flow of data from the source file to the agent's response.
 2. In a few sentences, explain how this agent-based, **retrieval-augmented approach** is more powerful and flexible than a traditional RPA workflow that might just filter a SQL table with predefined rules.
-

Part 2: Implementation (Coding)

Build a Python application that implements the design from Part 1. The application should create a simple command-line interface where a user can ask natural language questions and get back a list of relevant sales leads.

Your implementation should accomplish the following:

- Load lead data from the provided `leads.json` file.
- Apply a simple scoring logic by assigning a `"high_priority": true` flag to leads with job titles containing "VP" or "Director," or to companies with over 500 employees.
- Generate **vector embeddings** for the lead data. You are free to choose the embedding model (e.g., from a library like `sentence-transformers`).
- Store the leads and their vector embeddings in a **local vector database** of your choice (e.g., ChromaDB, FAISS).
- Implement a **conversational agent** that uses the vector database as a tool to find relevant leads. You may use a framework (e.g., LangChain, LlamaIndex) or build the core logic from scratch.

- Include a basic **memory mechanism** to allow the agent to handle simple follow-up questions.
 - Structure the code to be runnable via a single command (e.g., `python main.py`) with all dependencies listed in a `requirements.txt` file.
 - Incorporate basic error handling for operations like file loading or failed queries.
-

Part 3: Agent Evaluation & Improvement (Short Design - Optional)

In a brief writeup (less than half a page), propose a system for evaluating and improving the agent you designed. Address two key aspects:

- **Metrics:** What are the two most important metrics you would track to measure the agent's success, and why?
 - **Orchestration:** When would you add a second agent, and why?
 - **Improvement:** How would you collect user feedback on the agent's responses and use that feedback to improve its performance over time?
-

Deliverables

- All source code, including a `requirements.txt` file.
- Your workflow diagram from Part 1 (as a PNG, SVG, or in Markdown).
- A short document containing your written answers for Part 1 (and optional for Part 3).
- A `README.md` file with clear instructions on how to set up and run your project.